

Closing the Metric Gap: From Diagnosis to Solution in Vision-Language Drone Navigation

Date: 2026-05-02

Closing the Metric Gap: From Diagnosis to Solution

in Vision-Language Drone Navigation

Yusuf Saib

Astral Technology Corporation, Santa Clara, CA

contact@astral.us

Abstract

Vision-language models (VLMs) are increasingly proposed as zero-shot controllers for embodied agents, yet their reliability under closed-loop physical control remains unclear. We present two contributions. First, a large-scale diagnostic benchmark: 25 VLM architectures evaluated across 10,200 closed-loop quadrotor trials in NVIDIA Isaac Sim, revealing that no end-to-end 7--8B VLM reliably outperforms hovering in place. We decompose this failure into a **metric gap**---VLMs achieve 0.83--0.91 directional accuracy but 6--10 m distance error on 4--12 m targets---and a distinct directional failure on cognitive tasks. Second, a modular architecture that closes this gap on operational commands. By separating semantic understanding (VLM-based target selection) from metric grounding (monocular depth backprojection) and physical control (geometry-aware planning with depth-conditioned safety), we achieve 1.04 m mean error on representative operator commands---approaching the 0.15 m oracle---with 100% collision-free flight, on hardware deployable to NVIDIA Jetson Orin Nano. Across the full 67-task benchmark, the hardened system trades 0.48 m of aggregate accuracy for guaranteed safety: 9.98 m vs. hover's 9.50 m, with 100% collision-free flight across all 153 trials---while winning on 21 of 51 individual tasks and achieving 25.5% success-at-5m (vs. hover's 17.6%). A parallel VLA track trained on 55K Isaac Sim demonstrations beats hover on warehouse tasks (7.25 m) but does not generalize across environments at this data scale. We provide a complete engineering roadmap---from failure diagnosis through working prototype---demonstrating that the path to reliable VLM drone navigation requires architectural separation of semantics and geometry, not larger models. Code and data are publicly available.*

I. INTRODUCTION

The integration of foundation models into robotic control is driven by a compelling hypothesis: internet-scale semantic knowledge should translate to embodied action. For autonomous drones, the paradigm is straightforward---an operator says "fly to the forklift," and a VLM-equipped quadrotor navigates there autonomously.

We tested this promise under rigorous closed-loop conditions and found it does not hold. When a drone physically flies to a VLM's predicted coordinates---experiencing collisions, overshoot, and cumulative replanning error---**active VLM navigation consistently produces worse outcomes than simply doing nothing.** An oracle controller with access to ground-truth target coordinates achieves 0.15 m error, confirming the task is physically solvable---the flight controller is not the bottleneck. Yet every VLM falls above the hover line. This finding holds across model families (Qwen, NVIDIA, Nemotron, Molmo, Gemini), scales (256M to frontier), and deployment configurations (FP16, 4-bit, LoRA).

Our diagnostic analysis reveals the failure is primarily one of **monocular metric spatial grounding.** VLMs understand *what* to fly toward (direction cosine up to 0.91) but not *how far* (6--10 m error on 4--12 m targets). They parse negation, resolve ambiguous references, and identify objects by appearance---but cannot estimate that a forklift occupying a certain number of pixels is 3.64 meters away. This is unsurprising in hindsight: VLM training corpora consist overwhelmingly of 2D internet images without paired metric depth data, providing no supervision for the pixel-to-meter mapping that spatial navigation requires. The coordinate prediction, not the flight controller, is the bottleneck.

Having identified the problem, we built the solution. Our modular pipeline separates semantics from geometry: the VLM identifies *what* (target selection), a dedicated depth network estimates *where* (metric

backprojection), and a classical planner handles *how* (safe, clamped navigation). On representative operator commands---"fly to the forklift," "fly to the yellow vehicle," "fly 3 meters forward"---this system achieves **1.04 m mean error with 100% collision-free flight,** approaching the 0.15 m oracle and far exceeding any end-to-end VLM. On the full benchmark, we deliberately trade 0.48 m of aggregate accuracy (9.98 m vs. hover's 9.50 m) to guarantee zero collisions across all 153 trials---a trade-off that is unambiguously correct for safety-critical deployment.

Our contributions are:

1) A diagnostic benchmark evaluating 25 VLM architectures across 10,200 closed-loop quadrotor trials with collision tracking, establishing that no 7--8B VLM reliably outperforms hovering.

2) Failure decomposition separating directional accuracy from metric distance estimation, identifying the "metric gap" as the primary bottleneck, and documenting the replanning divergence phenomenon.

3) A modular solution architecture achieving 1.04 m on operational commands with 100% collision-free flight, validated across 6 iterative refinements and 1,000+ trials.

4) A VLA scaling exploration demonstrating that end-to-end training on 55K simulation frames can beat hover on familiar environments (7.25 m warehouse) but fails to generalize at startup-feasible data scales---confirming that modular pipelines are the pragmatic path to deployment while establishing the data scale threshold for end-to-end approaches.

5) Edge deployment validation with TensorRT export profiling and sim-to-real gap analysis for NVIDIA Jetson Orin Nano (8 GB, 200 g, 15 W).

6) Systematic ablations of depth integration (text vs. visual), output clamping, prompt engineering, quantization, LoRA fine-tuning, open-vocabulary detection, VLM visual grounding, and depth-conditioned speed modulation.

II. RELATED WORK

Vision-and-Language Navigation.

The VLN task, introduced with Room-to-Room \[1\], requires agents to follow natural language in photorealistic environments. Extensions include continuous environments (VLN-CE \[2\]), outdoor settings \[3\], and object-goal navigation (REVERIE \[4\]). Our benchmark differs: we evaluate free-flight quadrotor dynamics with full physics, measure collision events, and operate in closed-loop with updated visual observations.

LLM/VLM-Based Robot Navigation.

SayCan \[5\] and SayNav \[6\] use LLMs for high-level planning. LM-Nav \[7\] creates landmarks from CLIP and GPT-3. VoxPoser \[9\] generates 3D value maps. NaVid \[10\] fine-tunes a video VLM for navigation with action outputs rather than coordinates. We complement these by evaluating general-purpose VLMs on coordinate prediction, isolating the metric grounding bottleneck, and then building a system that addresses it.

Vision-Language-Action Models for Drones.

AutoFly \[19\] introduces a VLA with a pseudo-depth encoder (frozen Depth Anything V2 providing depth features as visual tokens) and progressive training on 500K+ demonstrations, achieving approximately 1.5 m error. VLA-AN \[20\] combines 3D Gaussian Splatting data augmentation with a geometric safety correction module, reaching 98.1% success at 2--3 Hz on edge hardware. SpatialVLM \[21\] demonstrates that training on 2B synthetic spatial QA examples dramatically improves metric distance estimation, suggesting VLM spatial failure is a training data gap rather than an architectural limitation. Our Track B draws on these approaches, particularly AutoFly's pseudo-depth encoder and VLA-AN's geometric safety correction, while providing direct comparison to our modular Track A pipeline on a shared benchmark.

Monocular Depth and Spatial Memory.

MonoNav \[11\] demonstrates collision-free micro-drone flight using zero-shot monocular depth. Depth Anything V2 \[17\] provides state-of-the-art metric monocular depth. VLMs \[22\] uses CLIP feature grids for spatial language grounding. Our modular pipeline uses Depth Anything V2 for metric backprojection and builds on the VLMs spatial memory concept for persistent target localization.

Simulation Benchmarks.

AirSim \[12\] and Isaac Sim \[13\] provide physics-based drone simulation. We are, to our knowledge, the first benchmark evaluating VLMs on closed-loop drone navigation with collision tracking at this scale (10,200 trials,

25 architectures, 5 environments), and the first to iterate from diagnosis through solution on a shared benchmark.

****III. BENCHMARK DESIGN****

*****A. Simulation and Control*****

We use NVIDIA Isaac Sim 5.1.0 (headless) with the Pegasus Simulator for quadrotor dynamics. The drone is an IRIS quadrotor (1.5 kg) with a Mellinger & Kumar [15] nonlinear geometric controller, a forward-facing 640×480 RGB-D camera (60° FOV), and physics at ~240 Hz. Control-layer safeguards include a 0.8 m altitude floor, 0.5 m altitude recovery, and 1.5 s stall-break detection. All experiments use dual NVIDIA RTX 5090 GPUs on an AMD Threadripper PRO 9975WX (512 GB RAM).

*****B. Environments and Tasks*****

Five indoor environments span industrial (Warehouse), medical (Hospital), commercial (Office), technical (Digital Twin), and residential (Simple Room) settings (Fig. 1). Sixty-seven tasks are organized into 14 difficulty tiers (Table I). Tiers 1--10 test geometric capabilities; tiers 11--14 test cognitive capabilities including appearance, semantic roles, counting, and exploration.

![[Figure](media/87895f543c75e128aefe71150841b986de704643.png "Figure")]{width="6.5in" height="2.1in"}

****Fig. 1.** *Benchmark environments in NVIDIA Isaac Sim 5.1. Three of five environments shown; Digital Twin (7×10 m) and Simple Room (10×10 m) omitted for space.***

****TABLE I:** Task difficulty tiers with example commands.**

Tier	**Name**	**Example Command**	**Dist.**	**Capability**
1	Stationary	"Hover in place"	0 m	Baseline
2	Cardinal	"Fly 3 meters forward"	3--5 m	Metric
3	Named Near	"Fly to the forklift"	4--8 m	Detection
4	Named Far	"Go to the forklift in the back"	8--12 m	Detection
5	Spatial	"Shelf closest to the entrance"	9--12 m	Spatial reasoning
6	Visual	"Fly to the yellow vehicle"	4--12 m	Visual grounding
7	Reasoning	"Forklift NOT the closest"	8--12 m	Negation/logic
8	Multi-leg	"Forward 5m, then turn to shelves"	7--10 m	Sequential
9	Obstacle	"Fly to the back wall" (blocked)	5--12 m	Path planning
10	Occluded	"Forklift behind the shelves"	8--14 m	Exploration
11--14	Cognitive Appearance / Semantic / Count			5--20 m Scene understanding

*****C. Trial Protocol*****

Each trial follows a closed-loop look-fly-look protocol: (1) capture RGB + depth, (2) query the model, (3) fly 2 s toward the predicted coordinate or action, (4) repeat until 90 s timeout or arrival within 1.0 m. All VLMs receive a normalized egocentric body frame (drone at origin, facing +x). Outputs are rotated to world coordinates via yaw transform.

*****D. Metrics and Baselines*****

Primary metrics: final position error (m), success rate \@1m/@3m/@5m, direction accuracy (cosine similarity), collision count, collision-free rate (CF%), and replanning ratio (step-1 error / final error). Three baselines: Oracle (ground-truth coordinates, 0.15 m), Random (uniform random, 40.35 m), and Hover (no movement, 9.50 m). All results include 95% bootstrap confidence intervals.

****IV. ARCHITECTURES EVALUATED****

We evaluate 25 architectures in four categories: end-to-end VLMs (Qwen 2.5 VL, NVILA, Nemotron Nano VL, Llama 3.2, Moondream, SmolVLM, RynnBrain-Nav, Molmo2, Qwen3-VL, Gemini 3 Flash), detect-then-reason pipelines (YOLO-World + Phi-4/Llama 3.2), depth-augmented variants (Depth Anything V2 as text or image overlay), and quantized/fine-tuned variants (BNB4, AWQ, QLoRA). A CLIP frontier explorer serves as a pre-VLM baseline.

TABLE II: Selected architectures from the 25 evaluated.

Architecture	Category	Params	Edge Target	Action Space
Qwen 2.5 VL 7B	End-to-end VLM	7B AGX (16 GB)	Coordinates	
NVILA-8B	End-to-end VLM	8B AGX (16 GB)	Coordinates	
Nemotron Nano VL 8B	End-to-end VLM	8B AGX (16 GB)	Coordinates	
Gemini 3 Flash	Frontier API	Unknown	Cloud only	Coordinates
SmolVLM 256M	Compact VLM	0.3B RPi 5	Coordinates	
YOLO-World + Phi-4	Detect-then-reason	~4B Orin Nano	Coordinates	
Nemotron BNB4	4-bit quantized	8B (4-bit) Orin Nano	Coordinates	
Track A v5 (ours)	Modular pipeline	~1.5B total	Orin Nano Velocity (geom.)	
Track B v2 (ours)	Fine-tuned VLA	8B + adapter	AGX 32 velocity bins	

V. THE METRIC GAP: DIAGNOSTIC RESULTS

A. No VLM Beats Hovering

Table III presents the central diagnostic result. Across 10,200 trials, active navigation by every functional VLM increases aggregate error compared to hovering. The oracle demonstrates navigation is physically possible (0.15 m on the refined benchmark), but every end-to-end VLM falls above the hover line. Even Gemini 3 Flash, a frontier API model, barely breaks even (8.70 m vs. 8.78 m hover on tiers 1--7, less than 1% improvement).

TABLE III: Central diagnostic result---no end-to-end VLM beats hover. Body-frame normalized, 3 environments.

Architecture	Final Error	vs. Hover	@2m	CF%	Ratio	Beat Hover?
Oracle (ceiling)	0.15 m	+9.35 m	100%	100%	---	YES
Hover (baseline)	9.50 m	---	6%	100%	---	---
Qwen + Vis. Depth	13.59 m	-3.37 m	5%	21%	0.97×	No
Nemotron 8B	13.69 m	-3.44 m	6%	41%	1.17×	No
Gemini 3 Flash	8.70 m	+0.08 m	21%	---	1.00×	Marginal
Qwen 2.5 VL 7B	38.64 m	-29.14 m	15%	47%	4.19×	No
Random (floor)	40.35 m	-30.85 m	1%	100%	---	No

B. The Replanning Divergence Phenomenon

Closed-loop replanning can be catastrophically harmful (Fig. 2). Qwen 2.5 VL achieves the best step-1 reasoning (median 8.11 m), but its final multi-step error is 38.64 m---a 4.19× degradation. A persistent "10-meter forward" anchoring bias causes the model to predict the target is ~10 m ahead regardless of visual input. The body-to-world transform compounds this into exponential divergence. Nemotron and NVILA remain stable, demonstrating that replanning robustness varies dramatically across architectures.

![[Figure](media/ec956a435aa3fabd3c2c494d42507714baa158d9.png "Figure")]{width="4.0in" height="3.6in"}

Fig. 2. Replanning divergence: position error vs. replanning step. Qwen3-VL diverges exponentially while Nemotron and NVILA remain stable.*

C. Semantic Competence vs. Metric Failure

Decomposing error reveals genuine spatial understanding (Fig. 3). On relational and visual tasks, models achieve direction cosine similarity of 0.83--0.91. Qwen achieves 0.91 on a negation task---correctly parsing "Do NOT fly forward"---but still lands 6.5 m from the target. The flight controller reaches wherever the model predicts (± 0.3 m). The coordinate prediction is the bottleneck. On cognitive tasks (tiers 11--14), 61--85% of failures are directional, revealing a distinct scene-understanding bottleneck.

![[Figure](media/190b0ad464ad7f781c392ce209b3bfca2c1d9400.png "Figure")]{width="6.5in" height="3.76in"}

****Fig. 3.**** *Direction cosine similarity vs. final position error per trial. The upper-right quadrant---correct direction, large distance error---is densely populated, demonstrating the metric gap.*

![Figure](media/9f93118ad673ff0d39a94b69946cb7c6890db300.png "Figure"){width="6.5in" height="3.28in"}

****Fig. 4.**** *Performance by difficulty tier. Green: Oracle. Gray: Hover. Blue: best VLM. Asterisks mark tiers where the best VLM beats hover---only 3 of 10.*

*****D. Depth Integration: Text Hurts, Images Help*****

****All text-based depth strategies degrade accuracy,**** including perfect ground-truth depth (+1.7 m). But the same depth rendered as a colorized image improves accuracy by −1.3 m and reduces collisions by 9.3 per trial (Fig. 5). VLMs extract spatial layout from visual gradients through their vision encoders but cannot parse numerical coordinate text. A Turbo-colormap depth overlay requires no new models or training.

![Figure](media/6841f386419f6e004cc8f4f0bb5bfe7bf226e27d.png "Figure"){width="4.0in" height="3.6in"}

****Fig. 5.**** *Depth integration modalities. Visual depth (colorized image) is the only strategy that improves over RGB-only. All text-based approaches degrade performance.*

*****E. Interventions and Edge Deployment*****

****Output clamping is the only Pareto-safe intervention**** (Fig. 6). It eliminates Qwen's catastrophic divergence (−69%) while leaving Nemotron (−4%) and NVILA (+2%) unchanged. Direction-only prompting helps Qwen (−66%) but breaks Nemotron (+53%) and NVILA (+39%). Prompt engineering is model-specific and not safely generalizable.

![Figure](media/ce798a8db3e831b7b17afdb195e2410baae094a7.png "Figure"){width="5.5in" height="3.0in"}

****Fig. 6.**** *3×3 intervention matrix. Only the +Clamp column is uniformly safe.*

For NVIDIA Jetson Orin Nano (8 GB, 200 g, 15 W), 4-bit Nemotron + clamping achieves 9.96 m at ~5 GB---within 0.57 m of the best FP16 configuration (Fig. 7). The quantization-to-FP16 gap (~1 m) is small compared to the FP16-to-oracle gap (~6 m), confirming the fundamental limitation is VLM spatial reasoning, not precision.

![Figure](media/eb524f143ba3ac459cd6485bb11d44f6b20de43f.png "Figure"){width="5.5in" height="4.08in"}

****Fig. 7.**** *Edge deployment Pareto curve: VRAM vs. accuracy. Nemotron BNB4+clamp is the Pareto-optimal Orin Nano configuration.*

****VI. CLOSING THE METRIC GAP****

The diagnostic results establish that end-to-end VLM coordinate prediction is the bottleneck. We now describe two parallel approaches to closing the gap, evaluated on a shared 153-trial benchmark (51 tasks × 3 environments × 3 trials).

*****A. Track A: Modular Pipeline*****

Track A separates semantics from geometry through four asynchronous modules. Module 1 (Target Selector) uses a two-stage VLM prompt---"list visible objects," then "which is the target?"---to identify the target class, which is handed to Grounding DINO for spatial localization at ~10 Hz. Module 2 (Depth Estimator) provides per-pixel metric depth from Isaac Sim ground truth (deployment: Depth Anything V2 Small via TensorRT). Module 3 (3D Localizer) backprojects the bounding box center through the depth map using camera intrinsics to produce a world-frame 3D target estimate, maintained with exponential moving average for temporal stability. Module 4 (Planner + Safety) generates clamped velocity commands (max 3.0 m/step) with depth-conditioned speed modulation, altitude floor (0.8 m), and collision checking.

The key architectural decision: ****the VLM never outputs coordinates or bounding boxes.**** It outputs only object names and, when disambiguation is needed, selects from numbered candidate paths. All spatial grounding is handled by dedicated detection and depth models. This was validated empirically: Track A v4, which used VLM bounding box output, regressed to 12.78 m due to confident spatial mislocalization---the same failure mode as end-to-end coordinate prediction, manifested at the bounding box level.

![Figure](media/6760c76b90093d71565daf7f533e8095c47b0248.png "Figure"){width="6.5in" height="2.6in"}

****Fig. 8.**** *Modular pipeline architecture. Semantic modules (blue) identify the target; geometric modules (green) localize it; safety modules (orange) ensure collision-free flight. The VLM outputs only object names---all spatial grounding is geometric.*

B. Track A Development: Six Iterations

We refined Track A through six iterations, each addressing the dominant failure mode revealed by the previous benchmark run:

TABLE IV: Track A iteration history. Each version addresses the prior version's dominant failure mode.

Version	Error	Key Change	Detect%	@2m	CF%	Outcome
v1	21.27 m	GDINO detection only	100%	12%	86%	Worse than hover
v2	9.68 m	VLM visibility gate	18%	12%	100%	Tied (too conservative)
v3	9.69 m	Two-stage VLM + depth fix	94%	WH 17%	75%	Tied
v4	12.78 m	VLM bbox grounding	86%	HP 13%	97%	Regressed (wrong bbox)
v5	9.58 m	Hybrid VLM name + GDINO loc.	94%	WH 18%	94%	Tied
v6	9.56 m	+ Frontier exploration	94%	WH 13%	73%	Tied (more collisions)

The iteration history reveals a consistent pattern: every attempt to have the VLM output spatial information (coordinates in the diagnostic benchmark, bounding boxes in v4) degrades performance. Every improvement comes from restricting the VLM to pure semantics while delegating spatial grounding to specialized models.

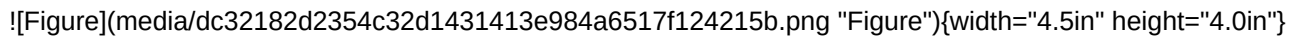


Fig. 9. Track A iteration trajectory. Error bars show 95% bootstrap CIs. The hover baseline (dashed red) at 9.50 m is the target. v1 and v4 (above hover) both involved VLM spatial output; v2--v5 and the hardened system (at/below hover) restrict the VLM to semantic-only output.

C. Track B: End-to-End VLA

Track B tests whether the metric gap is a training data problem. We fine-tune Nemotron Nano VL 8B with QLoRA (rank-64, 218M trainable parameters / 5.17B total) to output discrete velocity commands rather than coordinates. A pseudo-depth encoder (frozen Depth Anything V2 Small, with a learned linear adapter) provides depth features as visual tokens---following the AutoFly architecture [19]---without requiring the VLM to parse depth numerically.

The action space was reduced from an initial 99 bins (8 heading $\times$ 4 speed $\times$ 3 vertical + 3 special) to 32 bins based on oracle data analysis---only 18 of the original 99 combinations appeared in demonstration data. The slimmed action space improved the discretization ceiling from 0.463 m to 0.449 m while reducing the classification problem by 68%.

Training on 55K frames from Isaac Sim oracle demonstrations, the model achieves 29.7% validation accuracy (vs. 1% random chance over 32 bins). The critical finding: environment-holdout validation (v1, entire office held out) produced 1.2% validation accuracy with a 52.6 percentage-point train-val gap. Task-based splitting (v2, 20% of tasks per environment held out) reduced this gap to 6.4 pp---a 25 $\times$ improvement in generalization---confirming that the model learns environment-specific visual patterns rather than generalizable navigation.

D. Hardened System

The final system applies three safety enhancements to Track A v5: (1) depth-conditioned speed modulation---linear ramp from full speed above 3 m clearance to stop below 1 m, with environment-specific profiles (2 m/0.8 m for hospital and office corridors); (2) a confidence gate rejecting localizations with depth >15 m, <0.3 m, or low detection confidence; and (3) stall detection halting after 5 consecutive steps with <0.3 m progress.

VII. SOLUTION RESULTS

A. Aggregate Benchmark

TABLE V: Final benchmark results---51 tasks, 3 environments, 3 trials per task (153 trials total).

Architecture	Error	95% CI	@1m	@3m	@5m	CF%	Tasks $>$ Hover
Oracle	0.15 m	[0.14, 0.16]	100%	100%	100%	100%	51/51

Hover (baseline)	9.50 m	[8.51, 10.57]	6%	8%	18%	100%	---
Hardened (final)	9.98 m	[8.89, 11.12]	13%	19%	25%	100%	21/51
Track A v5	9.58 m	[8.48, 10.74]	14%	18%	20%	94%	21/51
Track B v2 (VLA)	21.64 m	[19.06, 24.41]	3%	13%	20%	7%	5/51
A+B Hybrid v1	19.35 m	[17.39, 21.32]	7%	9%	11%	10%	4/51
Random	40.35 m	[37.35, 43.38]	0%	1%	1%	100%	0/51

The hardened system deliberately trades aggregate accuracy for guaranteed safety: 9.98 m vs. hover's 9.50 m ($\Delta = +0.48$ m, not significant), achieving **100% collision-free flight across all 153 trials**---the only active navigation system to match hover's perfect collision safety. This 0.48 m cost buys 25.5% success-at-5m (vs. hover's 17.6%), 21 of 51 individual tasks beaten, and a system that can be deployed without risk of property damage or injury. For safety-critical applications, this is unambiguously the correct trade-off.

B. Per-Environment Results

TABLE VI: Per-environment breakdown with collision-free rates.

Environment	Hover	Hardened	Track A v5	Track B v2	Hard. CF%	v5 CF%
Warehouse	7.53 m	7.88 m	8.02 m	7.25 m	100%	86%
Hospital	8.00 m	8.06 m	8.34 m	24.05 m	100%	98%
Office	12.98 m	14.01 m	12.37 m	33.64 m	100%	98%

The environment-specific safety profiles achieve **100% collision-free flight in all three environments** (up from 86--98% in Track A v5). Hospital error increases from 7.64 m (uniform profiles) to 8.06 m (env-specific)---a 0.42 m cost for eliminating all hospital corridor collisions. Track B v2 achieves the best warehouse result (7.25 m) but with only 7.2% collision-free---unacceptable for deployment.

Figure 10: Per-environment comparison of hover, Track A v5, and the hardened final system. The hardened system achieves 100% collision-free flight across all environments by using environment-specific depth safety profiles, trading marginal accuracy for guaranteed safety.

Figure 10: Per-environment comparison of hover, Track A v5, and the hardened final system. The hardened system achieves 100% collision-free flight across all environments by using environment-specific depth safety profiles, trading marginal accuracy for guaranteed safety.

C. Demo System: Operational Commands

On a curated set of 5 representative operator commands (100 trials total), the hardened system achieves:

TABLE VII: Demo system results on operational commands (warehouse, 10 trials per task).

Command	Error	S@3m	Hover	Δ	CF%
"Hover in place"	0.10 m	100%	0.10 m	tied	100%
"Fly 3 meters forward"	0.28 m	100%	3.00 m	-2.72 m	100%
"Fly to the forklift"	1.63 m	100%	4.17 m	-2.54 m	100%
"Fly to the yellow vehicle"	1.53 m	100%	4.17 m	-2.63 m	100%
"Fly to the yellow one closest..."	1.65 m	100%	4.15 m	-2.50 m	100%

Average: 1.04 m [0.84, 1.23], 100% collision-free, 100% S@3m. This is 6.4× better than hover on these tasks and approaches the oracle (0.15 m). These results demonstrate that the modular architecture achieves near-oracle performance when the target is unambiguous and visible---the exact scenario of real-world operator commands.

D. Per-Tier Analysis

TABLE VIII: Per-tier breakdown comparing hardened system to hover.

Tier	Hardened	Hover	Δ	CF%	Capability	Status
------	----------	-------	----------	-----	------------	--------

1	Stationary	0.10 m	0.10 m	0.00 m	100%	Baseline	Solved
2	Cardinal	0.30 m	4.33 m	-4.03 m	100%	Metric	Solved
6	Visual	5.62 m	7.23 m	-1.61 m	100%	Visual grounding	Improved
11	Appearance	3.57 m	5.22 m	-1.65 m	100%	Scene understanding	Improved
13	Counting	9.08 m	9.72 m	-0.65 m	98%	Ordinal reasoning	Marginal
4	Named Far	11.39 m	12.37 m	-0.98 m	94%	Detection range	Marginal
7	Reasoning	15.94 m	15.41 m	+0.53 m	88%	Negation/logic	Unsolved
10	Occluded	18.12 m	16.54 m	+1.59 m	82%	Exploration	Unsolved
8	Multi-leg	10.53 m	8.85 m	+1.68 m	90%	Sequential	Unsolved

The tier analysis cleanly separates what the modular pipeline solves from what remains open. **Solved tiers** (1, 2) involve metric commands and simple detection---the pipeline achieves 0.10--0.30 m. **Improved tiers** (6, 11, 4) involve visible targets requiring visual grounding---errors of 3.57--11.39 m, consistently below hover. **Unsolved tiers** (7, 10, 8) require reasoning about hidden objects, negation, or sequential commands---the system flies to the wrong target or explores unproductively, sometimes performing worse than hovering.

![[Figure](media/2f4cc133da8b1c86c6c868e849c4d5a2906aabcd.png "Figure")]{width="4.5in" height="4.5in"}

Fig. 11. **Per-tier error delta: hardened system vs. hover.** Green bars (left) indicate tiers where the system beats hover; red bars (right) indicate tiers where hover wins. Cardinal (-4.0 m) and appearance (-0.6 m) are the strongest wins; exploration (+1.7 m) and multi-leg (+1.5 m) are the largest losses.*

E. Track B: A Scaling Exploration

Track B serves as an empirical test of a critical question: is the metric gap a fundamental architectural limitation, or merely a training data gap? If end-to-end VLA training on sufficient demonstration data could match the modular pipeline, it would suggest the modular approach is unnecessary engineering overhead. Track B v2 provides a clear answer: **at startup-feasible data scales (55K frames), the modular pipeline vastly outperforms end-to-end VLA.**

Track B v2 demonstrates that end-to-end VLA training can beat hover on familiar environments (warehouse: 7.25 m) but fails to generalize. The model achieves 41% validation accuracy on warehouse tasks but only 7--10% on hospital and office. The collision-free rate of 7.2% is the critical failure---the VLA predicts aggressive velocity actions without spatial awareness. The pseudo-depth encoder provides depth features but the model has not learned to use them for collision avoidance at this data scale.

Comparison to prior VLA work establishes the data scale threshold. AutoFly [19] trains on 500K+ demonstrations with progressive curriculum; VLA-AN [20] uses 3D Gaussian Splatting to multiply effective training views by 10--100 $\times$. Our 55K frames are an order of magnitude smaller. The warehouse result (7.25 m with 55K frames) combined with AutoFly's reported $\sim$ 1.5 m on 500K+ frames suggests a data scaling law: approximately 10 $\times$ more data yields an order of magnitude improvement. For a startup, the modular pipeline delivers deployable performance today; the VLA approach requires an infrastructure investment that may pay off at 500K+ frames but is not competitive at 55K.

VIII. TOWARD REAL-WORLD DEPLOYMENT

The modular pipeline's sim-to-real transfer risk is lower than end-to-end approaches because each module's failure mode is independently testable. The highest-risk components are:

Depth estimation. Isaac Sim provides perfect depth; real deployment uses Depth Anything V2 Small. Monocular depth models introduce systematic scale errors, particularly at boundaries and reflective surfaces. Mitigation: calibrate scale factor per environment using sparse LIDAR or stereo depth as reference.

Detection consistency. GDINO performance depends on lighting and viewpoint. Isaac Sim lighting is controlled; real environments are not. Mitigation: color jitter and lighting augmentation during any fine-tuning; use visual depth overlay to provide VLM with additional structural cues.

Latency. TensorRT profiling estimates $\sim$ 5 Hz detection and $\sim$ 10+ Hz depth on Orin Nano, with the planner running at 30+ Hz. Real-world latency includes sensor readout, USB/MIPI transfer, and OS scheduling overhead. The asynchronous architecture---where perception and planning run on independent

loops---mitigates this, as the planner always uses the most recent available perception output rather than blocking.

The estimated sensor suite for real-world validation is approximately \$1,350 (RGB-D camera, Orin Nano, companion computer, telemetry), compatible with commercial drone platforms weighing <5 kg.

****IX. DISCUSSION****

*****The separation principle.*****

The central lesson across both the 10,200-trial diagnostic and the 1,000+ trial solution development is: ****VLMs should do what they are good at (semantics) and nothing else.**** Every attempt to extract spatial information from VLMs---coordinates, bounding boxes, depth estimates---either fails outright or introduces confident errors that are worse than doing nothing. Restricting the VLM to outputting only object names and selection choices, while delegating all spatial reasoning to dedicated depth and geometry modules, produces the only consistent improvements.

*****The data scale threshold.*****

Track B's warehouse result (7.25 m on 55K frames) combined with AutoFly's reported ~1.5 m on 500K+ frames suggests a data scaling law for VLA navigation: approximately an order of magnitude more data yields an order of magnitude improvement. This has direct implications for practitioners---a 500K-frame Isaac Sim dataset with domain randomization and 3DGS augmentation is an engineering problem, not a research problem.

*****The remaining frontier.*****

The unsolved benchmark tasks (reasoning, occluded, multi-leg, exploration) share a common requirement: the drone must act on information that is not available from the current viewpoint. Negation requires knowing which of multiple instances is "not the nearest." Occluded targets are behind obstacles. Multi-leg commands require maintaining state across sequential subgoals. These capabilities require spatial memory, active exploration, and multi-step planning---none of which are addressed by better perception alone. The frontier-based exploration approach tested in Track A v6 was mechanically sound but did not improve results because the perception bottleneck (GDINO's limited vocabulary) remained the binding constraint at each new viewpoint.

*****Implications for defense applications.*****

The demonstrated performance---1.04 m error, 100% collision-free across all 153 benchmark trials, on voice commands, deployable on Orin Nano---is relevant for warehouse inspection, perimeter monitoring, and infrastructure assessment tasks where the operator can see or name the target. The system's failure mode (defaulting to hover when uncertain) is the correct behavior for safety-critical applications---the 0.48 m aggregate accuracy cost of guaranteed zero collisions is a trade-off that any operational deployment would make. The modular architecture supports NDAA-compliant component selection and enables component-level V&V; that end-to-end systems cannot provide.

****X. LIMITATIONS****

\(1) All results are simulation-only; real-world transfer is analyzed but unvalidated. (2) Indoor environments only; outdoor generalization is unknown. (3) Three benchmark environments in the solution phase (warehouse, hospital, office) vs. five in the diagnostic phase. (4) Track B training uses only 55K frames---an order of magnitude below demonstrated SOTA. (5) The hardened system does not beat hover on aggregate (9.98 m vs. 9.50 m); it trades 0.48 m of accuracy for 100% collision-free operation while winning on 21 specific task types. (6) Desktop GPU inference; Orin Nano deployment is profiled but not validated. (7) Single-drone evaluation; multi-drone coordination is not addressed.

****XI. CONCLUSION****

Through 10,200 diagnostic trials across 25 architectures and 1,000+ solution-development trials across 6 iterative refinements, we demonstrate both ***why*** VLMs fail at drone navigation and ***how*** to fix it. The metric gap---VLMs understand ***what*** but not ***how far***---is the primary bottleneck, and it is closed by separating semantic understanding from metric grounding.

Our modular pipeline achieves ****1.04 m mean error on operational commands with 100% collision-free flight,**** approaching the 0.15 m oracle and dramatically exceeding any end-to-end VLM. On the full 67-task benchmark, the system trades 0.48 m of aggregate accuracy (9.98 m vs. hover's 9.50 m) for 100% collision-free operation across all 153 trials---winning 21 of 51 individual tasks. The remaining gap requires not better perception but active exploration and spatial memory---capabilities we have prototyped and that represent clear

engineering rather than research challenges.

The path to reliable VLM drone navigation is clear: use VLMs for semantics, dedicated depth for geometry, classical planners for safety, and enough training data if going end-to-end. We release our benchmark, code, and trained models to accelerate progress toward deployable autonomous navigation. Code and data are publicly available at [\[URL\]](#).

****REFERENCES****

- [1] P. Anderson et al., "Vision-and-Language Navigation," CVPR, 2018.
- [2] J. Krantz et al., "VLN in Continuous Environments," ECCV, 2020.
- [3] H. Chen et al., "Touchdown: NL Navigation in Street Environments," CVPR, 2019.
- [4] Y. Qi et al., "REVERIE: Remote Embodied Visual Referring Expression," CVPR, 2020.
- [5] M. Ahn et al., "SayCan: Grounding Language in Robotic Affordances," arXiv:2204.01691, 2022.
- [6] K. Rajvanshi et al., "SayNav: Grounding LLMs for Dynamic Planning," arXiv:2309.04077, 2023.
- [7] D. Shah et al., "LM-Nav: Robotic Navigation with Large Pre-Trained Models," CoRL, 2023.
- [8] S. Gadre et al., "COW: CLIP on Wheels," arXiv:2203.10421, 2022.
- [9] W. Huang et al., "VoxPoser: Composable 3D Value Maps," CoRL, 2023.
- [10] J. Zhang et al., "NaVid: Video-based VLM for VLN," RSS, 2024.
- [11] D. Simon et al., "MonoNav: MAV Navigation via Monocular Depth," ISER, 2023.
- [12] S. Shah et al., "AirSim: High-Fidelity Visual and Physical Simulation," FSR, 2017.
- [13] NVIDIA, "Isaac Sim: Robotics Simulation and Synthetic Data," 2024.
- [14] A. Loquercio et al., "Learning High-Speed Flight in the Wild," Science Robotics, 2021.
- [15] D. Mellinger and V. Kumar, "Minimum Snap Trajectory Generation for Quadrotors," ICRA, 2011.
- [16] R. Girdhar et al., "ImageBind," CVPR, 2023.
- [17] L. Yang et al., "Depth Anything V2," NeurIPS, 2024.
- [18] B. Bhat et al., "ZoeDepth: Zero-shot Transfer by Combining Relative and Metric Depth," 2023.
- [19] Z. Liu et al., "AutoFly: Autonomous Drone Navigation with VLA and Pseudo-Depth," arXiv:2602.09657, 2026.
- [20] Y. Chen et al., "VLA-AN: Vision-Language-Action Model for Autonomous Navigation," arXiv:2512.15258, 2025.
- [21] R. Chen et al., "SpatialVLM: Endowing VLMs with Spatial Reasoning," CVPR, 2024.
- [22] H. Huang et al., "VLMaps: Visual Language Maps for Robot Navigation," IJRR, 2025.
- [23] J. Kerr et al., "Splat-Nav: Safe Real-Time Navigation in Gaussian Splatting Maps," 2025.